

在 Go 语言项目中使用 Zap 日志库

本文先介绍了 Go 语言原生的日志库的使用，然后详细介绍了非常流行的 Uber 开源的 Zap 日志库，同时介绍了如何搭配 Lumberjack 实现日志的切割和归档。

介绍

在许多 Go 语言项目中，我们需要一个好的日志记录器能够提供下面这些功能：

- 能够将事件记录到文件中，而不是应用程序控制台。
- 日志切割 - 能够根据文件大小、时间或间隔等来切割日志文件。
- 支持不同的日志级别。例如 **INFO**，**DEBUG**，**ERROR** 等。
- 能够打印基本信息，如调用文件/函数名和行号，日志时间等。

默认的 Go Logger

在介绍 Uber-go 的 Zap 包之前，让我们先看看 Go 语言提供的基本日志功能。Go 语言提供的默认日志包是 **log**。

实现 Go Logger

实现一个 Go 语言中的日志记录器非常简单——创建一个新的日志文件，然后设置它为日志的输出位置。

设置 Logger

我们可以像下面的代码一样设置日志记录器：

```
func SetupLogger() {
    logFileLocation, _ := os.OpenFile("/Users/q1mi/test.log",
os.O_CREATE|os.O_APPEND|os.O_RDWR, 0744)
    log.SetOutput(logFileLocation)
}
```

使用 Logger

让我们来写一些虚拟的代码来使用这个日志记录器。

在当前的示例中，我们将建立一个到 URL 的 HTTP 连接，并将状态代码/错误记录到日志文件中。

```
func simpleHttpGet(url string) {
    resp, err := http.Get(url)
    if err != nil {
        log.Printf("Error fetching url %s : %s", url, err.Error())
    } else {
        log.Printf("Status Code for %s : %s", url, resp.Status)
    }
}
```

```
    resp.Body.Close()
  }
}
```

Logger 的运行

现在让我们执行上面的代码并查看日志记录器的运行情况。

```
func main() {
    SetupLogger()
    simpleHttpGet("www.google.com")
    simpleHttpGet("http://www.google.com")
}
```

当我们执行上面的代码，我们能看到一个 `test.log` 文件被创建，下面的内容会被添加到这个日志文件中。

```
2019/05/24 01:14:13 Error fetching url www.google.com : Get www.google.com:
unsupported protocol scheme ""
2019/05/24 01:14:14 Status Code for http://www.google.com : 200 OK
```

Go Logger 的优势和劣势

优势

它最大的优点是使用非常简单。我们可以设置任何 `io.Writer` 作为日志记录输出并向其发送要写入的日志。

劣势

- 仅限基本的日志级别
- 只有一个 `Print` 选项。不支持 `INFO/DEBUG` 等多个级别。
- 对于错误日志，它有 `Fatal` 和 `Panic`
 - `Fatal` 日志通过调用 `os.Exit(1)` 来结束程序
 - `Panic` 日志在写入日志消息之后抛出一个 `panic`
- 但是它缺少一个 `ERROR` 日志级别，这个级别可以在不抛出 `panic` 或退出程序的情况下记录错误
- 缺乏日志格式化的能力——例如记录调用者的函数名和行号，格式化日期和时间格式等等。
- 不提供日志切割的能力。

Uber-go Zap

Zap 是非常快的、结构化的，分日志级别的 Go 日志库。

为什么选择 Uber-go Zap

- 它同时提供了结构化日志记录和 `printf` 风格的日志记录
- 它非常的快

根据 Uber-go Zap 的文档，它的性能比类似的结构化日志包更好——也比标准库更快。以下是 Zap 发布的基准测试信息：

记录一条消息和 10 个字段

Package	Time	Time % to zap	Objects Allocated
⚡ zap	862 ns/op	+0%	5 allocs/op
⚡ zap (sugared)	1250 ns/op	+45%	11 allocs/op
zerolog	4021 ns/op	+366%	76 allocs/op
go-kit	4542 ns/op	+427%	105 allocs/op
apex/log	26785 ns/op	+3007%	115 allocs/op
logrus	29501 ns/op	+3322%	125 allocs/op
log15	29906 ns/op	+3369%	122 allocs/op

记录一个静态字符串，没有任何上下文或 printf 风格的模板

Package	Time	Time % to zap	Objects Allocated
⚡ zap	118 ns/op	+0%	0 allocs/op
⚡ zap (sugared)	191 ns/op	+62%	2 allocs/op
zerolog	93 ns/op	-21%	0 allocs/op
go-kit	280 ns/op	+137%	11 allocs/op
standard library	499 ns/op	+323%	2 allocs/op
apex/log	1990 ns/op	+1586%	10 allocs/op
logrus	3129 ns/op	+2552%	24 allocs/op
log15	3887 ns/op	+3194%	23 allocs/op

安装

运行下面的命令安装 Zap：

```
go get -u go.uber.org/zap
```

配置 Zap Logger

Zap 提供了两种类型的日志记录器——**Sugared Logger** 和 **Logger**。

- 在性能很好但不是很关键的上下文中，使用 **SugaredLogger**。它比其他结构化日志记录包快 4-10 倍，并且支持结构化和 **printf** 风格的日志记录。

- 在每一微秒和每一次内存分配都很重要的上下文中，使用 `Logger`。它甚至比 `SugaredLogger` 更快，内存分配次数也更少，但它只支持强类型的结构化日志记录。

Logger

通过调用 `zap.NewProduction()` / `zap.NewDevelopment()` 或者 `zap.Example()` 创建一个 `Logger`。

上面的每一个函数都将创建一个 logger。唯一的区别在于它将记录的信息不同。例如 production logger 默认记录调用函数信息、日期和时间等。

通过 `Logger` 调用 `Info` / `Error` 等。默认情况下日志都会打印到应用程序的 console 界面。

```
var logger *zap.Logger

func main() {
    InitLogger()
    defer logger.Sync()
    simpleHttpGet("www.google.com")
    simpleHttpGet("http://www.google.com")
}

func InitLogger() {
    logger, _ = zap.NewProduction()
}

func simpleHttpGet(url string) {
    resp, err := http.Get(url)
    if err != nil {
        logger.Error(
            "Error fetching url..",
            zap.String("url", url),
            zap.Error(err))
    } else {
        logger.Info("Success..",
            zap.String("statusCode", resp.Status),
            zap.String("url", url))
        resp.Body.Close()
    }
}
```

在上面的代码中，我们首先创建了一个 `Logger`，然后使用 `Info` / `Error` 等 `Logger` 方法记录消息。

日志记录器方法的语法是这样的：

```
func (log *Logger) MethodXXX(msg string, fields ...Field)
```

其中 `MethodXXX` 是一个可变参数函数，可以是 `Info` / `Error` / `Debug` / `Panic` 等。每个方法都接受一个消息字符串和任意数量的 `zapcore.Field` 场参数。

每个 `zapcore.Field` 其实就是一组键值对参数。

我们执行上面的代码会得到如下输出结果：

```
{
  "level": "error",
  "ts": 1572159218.912792,
  "caller": "zap_demo/temp.go:25",
  "msg": "Error fetching url..",
  "url": "www.sogo.com",
  "error": "Get www.sogo.com: unsupported protocol scheme\n\"",
  "stacktrace": "main.simpleHttpGet\n\t/Users/q1mi/zap_demo/temp.go:25\nmain.main\n\t/Users/q1mi/zap_demo/temp.go:14\nruntime.main\n\t/usr/local/go/src/runtime/proc.go:203"
}
{
  "level": "info",
  "ts": 1572159219.1227388,
  "caller": "zap_demo/temp.go:30",
  "msg": "Success..",
  "statusCode": "200 OK",
  "url": "http://www.sogo.com"
}
```

Sugared Logger

现在让我们使用 Sugared Logger 来实现相同的功能。

- 大部分的实现基本都相同。
- 惟一的区别是，我们通过调用主 logger 的 `.Sugar()` 方法来获取一个 `SugaredLogger`。
- 然后使用 `SugaredLogger` 以 `printf` 格式记录语句。

下面是修改过后使用 `SugaredLogger` 代替 `Logger` 的代码：

```
var sugarLogger *zap.SugaredLogger

func main() {
    InitLogger()
    defer sugarLogger.Sync()
    simpleHttpGet("www.google.com")
    simpleHttpGet("http://www.google.com")
}

func InitLogger() {
    logger, _ := zap.NewProduction()
    sugarLogger = logger.Sugar()
}

func simpleHttpGet(url string) {
    sugarLogger.Debugf("Trying to hit GET request for %s", url)
    resp, err := http.Get(url)
    if err != nil {
        sugarLogger.Errorf("Error fetching URL %s : Error = %s", url, err)
    } else {
        sugarLogger.Infof("Success! statusCode = %s for URL %s", resp.Status, url)
        resp.Body.Close()
    }
}
```

当你执行上面的代码会得到如下输出：

```
{ "level": "error", "ts": 1572159149.923002, "caller": "logic/temp2.go:27", "msg": "Error fetching URL www.sogo.com : Error = Get www.sogo.com: unsupported protocol scheme \"\"\", \"stacktrace\": \"main.simpleHttpGet\\n\\t/Users/q1mi/zap_demo/logic/temp2.go:27\\nmain.main\\n\\t/Users/q1mi/zap_demo/logic/temp2.go:14\\nruntime.main\\n\\t/usr/local/go/src/runtime/proc.go:203\" }
{ "level": "info", "ts": 1572159150.192585, "caller": "logic/temp2.go:29", "msg": "Success ! statusCode = 200 OK for URL http://www.sogo.com" }
```

你应该注意到了，到目前为止这两个 logger 都打印输出 JSON 结构格式。

在本博客的后面部分，我们将更详细地讨论 `SugaredLogger`，并了解如何进一步配置它。

定制 Logger

将日志写入文件而不是终端

我们要做的第一个更改是把日志写入文件，而不是打印到应用程序控制台。

我们将使用 `zap.New(...)` 方法来手动传递所有配置，而不是使用像 `zap.NewProduction()` 这样的预置方法来创建 logger。

```
func New(core zapcore.Core, options ...Option) *Logger
```

`zapcore.Core` 需要三个配置——**Encoder**，**WriteSyncer**，**LogLevel**。

1. **Encoder**：编码器（如何写入日志）。我们将使用开箱即用的 `NewJSONEncoder()`，并使用预先设置的 `ProductionEncoderConfig()`。

```
zapcore.NewJSONEncoder(zap.NewProductionEncoderConfig())
```

2. **WriterSyncer**：指定日志将写到哪里去。我们使用 `zapcore.AddSync()` 函数并且将打开的文件句柄传进去。

```
file, _ := os.Create("./test.log")
writeSyncer := zapcore.AddSync(file)
```

3. **Log Level**：哪种级别的日志将被写入。

我们将修改上述部分中的 Logger 代码，并重写 `InitLogger()` 方法。其余方法——`main()` / `SimpleHttpGet()` 保持不变。

```
func InitLogger() {
    writeSyncer := getLogWriter()
    encoder := getEncoder()
    core := zapcore.NewCore(encoder, writeSyncer, zapcore.DebugLevel)

    logger := zap.New(core)
    sugarLogger = logger.Sugar()
}

func getEncoder() zapcore.Encoder {
    return zapcore.NewJSONEncoder(zap.NewProductionEncoderConfig())
}

func getLogWriter() zapcore.WriteSyncer {
    file, _ := os.Create("./test.log")
    return zapcore.AddSync(file)
}
```

当使用这些修改过的 logger 配置调用上述部分的 `main()` 函数时，以下输出将打印在文件 `test.log` 中：

```
{ "level": "debug", "ts": 1572160754.994731, "msg": "Trying to hit GET request for www.sogo.com" }
{ "level": "error", "ts": 1572160754.994982, "msg": "Error fetching URL www.sogo.com : Error = Get www.sogo.com: unsupported protocol scheme \"\""}
{ "level": "debug", "ts": 1572160754.994996, "msg": "Trying to hit GET request for http://www.sogo.com" }
{ "level": "info", "ts": 1572160757.3755069, "msg": "Success! statusCode = 200 OK for URL http://www.sogo.com" }
```

将 JSON Encoder 更改为普通的 Log Encoder

现在，我们希望将编码器从 JSON Encoder 更改为普通 Encoder。为此，我们需要将 `NewJSONEncoder()` 更改为 `NewConsoleEncoder()`。

```
return zapcore.NewConsoleEncoder(zap.NewProductionEncoderConfig())
```

当使用这些修改过的 logger 配置调用上述部分的 `main()` 函数时，以下输出将打印在文件 `test.log` 中：

```
1.572161051846623e+09  debug  Trying to hit GET request for www.sogo.com
1.572161051846828e+09  error  Error fetching URL www.sogo.com : Error = Get
www.sogo.com: unsupported protocol scheme ""
1.5721610518468401e+09  debug  Trying to hit GET request for http://www.sogo.com
1.572161052068744e+09  info   Success! statusCode = 200 OK for URL
http://www.sogo.com
```

更改时间编码并添加调用者详细信息

鉴于我们对配置所做的更改，有下面两个问题：

1. 时间是以非人类可读的方式展示，例如 `1.572161051846623e+09`
2. 调用方函数的详细信息没有显示在日志中

我们要做的第一件事是覆盖默认的 `ProductionConfig()`，并进行以下更改：

- 修改时间编码器
- 在日志文件中使用大写字母记录日志级别

```
func getEncoder() zapcore.Encoder {  
    encoderConfig := zap.NewProductionEncoderConfig()  
    encoderConfig.EncodeTime = zapcore.ISO8601TimeEncoder  
    encoderConfig.EncodeLevel = zapcore.CapitalLevelEncoder  
    return zapcore.NewConsoleEncoder(encoderConfig)  
}
```

接下来，我们将修改 zap logger 代码，添加将调用函数信息记录到日志中的功能。为此，我们将在 `zap.New(...)` 函数中添加一个 `Option`。

```
logger := zap.New(core, zap.AddCaller())
```

当使用这些修改过的 logger 配置调用上述部分的 `main()` 函数时，以下输出将打印在文件 `test.log` 中：

```
2019-10-27T15:33:29.855+0800    DEBUG    logic/temp2.go:47    Trying to hit GET  
request for www.sogo.com  
2019-10-27T15:33:29.855+0800    ERROR    logic/temp2.go:50    Error fetching URL  
www.sogo.com : Error = Get www.sogo.com: unsupported protocol scheme ""  
2019-10-27T15:33:29.856+0800    DEBUG    logic/temp2.go:47    Trying to hit GET  
request for http://www.sogo.com  
2019-10-27T15:33:30.125+0800    INFO     logic/temp2.go:52    Success! statusCode =  
200 OK for URL http://www.sogo.com
```

AddCallerSkip

当我们不是直接使用初始化好的 logger 实例记录日志，而是将其包装成一个函数等，此时日志记录的函数调用链会增加，想要获得准确的调用信息就需要通过 `AddCallerSkip` 函数来跳过。

```
logger := zap.New(core, zap.AddCaller(), zap.AddCallerSkip(1))
```

将日志输出到多个位置

我们可以将日志同时输出到文件和终端。

```
func getLogWriter() zapcore.WriteSyncer {
    file, _ := os.Create("./test.log")
    // 利用 io.MultiWriter 支持文件和终端两个输出目标
    ws := io.MultiWriter(file, os.Stdout)
    return zapcore.AddSync(ws)
}
```

将 err 日志单独输出到文件

有时候我们除了将全量日志输出到 `xx.log` 文件中之外，还希望将 `ERROR` 级别的日志单独输出到一个名为 `xx.err.log` 的日志文件中。我们可以通过以下方式实现。

```
func InitLogger() {
    encoder := getEncoder()
    // test.log 记录全量日志
    logF, _ := os.Create("./test.log")
    c1 := zapcore.NewCore(encoder, zapcore.AddSync(logF), zapcore.DebugLevel)
    // test.err.log 记录 ERROR 级别的日志
    errF, _ := os.Create("./test.err.log")
    c2 := zapcore.NewCore(encoder, zapcore.AddSync(errF), zapcore.ErrorLevel)
    // 使用 NewTee 将 c1 和 c2 合并到 core
    core := zapcore.NewTee(c1, c2)
    logger = zap.New(core, zap.AddCaller())
}
```

使用 Lumberjack 进行日志切割归档

这个日志程序中唯一缺少的就是日志切割归档功能。

Zap 本身不支持切割归档日志文件。

官方的说法是为了添加日志切割归档功能，我们将使用第三方库 Lumberjack 来实现。

目前只支持按文件大小切割，原因是按时间切割效率低且不能保证日志数据不被破坏。详情戳 <https://github.com/natefinch/lumberjack/issues/54>。

想按日期切割可以使用 github.com/lestrrat-go/file-rotatelogs 这个库，虽然目前不维护了，但也够用了。

```
// 使用 file-rotatelogs 按天切割日志
import rotatelogs "github.com/lestrrat-go/file-rotatelogs"

l, _ := rotatelogs.New(
    filename+".%Y%m%d%H%M",
    rotatelogs.WithMaxAge(30*24*time.Hour), // 最长保存30天
```

```
    rotatelog.WithRotationTime(time.Hour*24), // 24小时切割一次
)
zapcore.AddSync(1)
```

安装

执行下面的命令安装 Lumberjack v2 版本。

```
go get gopkg.in/natefinch/lumberjack.v2
```

Zap Logger 中加入 Lumberjack

要在 Zap 中加入 Lumberjack 支持，我们需要修改 `WriteSyncer` 代码。我们将按照下面的代码修改 `getLogWriter()` 函数：

```
func getLogWriter() zapcore.WriteSyncer {
    lumberjackLogger := &lumberjack.Logger{
        Filename:   "./test.log",
        MaxSize:    10,
        MaxBackups: 5,
        MaxAge:     30,
        Compress:   false,
    }
    return zapcore.AddSync(lumberjackLogger)
}
```

Lumberjack Logger 采用以下属性作为输入：

- `Filename`：日志文件的位置
- `MaxSize`：在进行切割之前，日志文件的最大大小（以 MB 为单位）
- `MaxBackups`：保留旧文件的最大个数
- `MaxAge`：保留旧文件的最大天数
- `Compress`：是否压缩/归档旧文件

测试所有功能

最终，使用 Zap/Lumberjack logger 的完整示例代码如下：

```
package main

import (
    "net/http"

    "gopkg.in/natefinch/lumberjack.v2"
    "go.uber.org/zap"
)
```

```
"go.uber.org/zap/zapcore"

)

var sugarLogger *zap.SugaredLogger

func main() {
    InitLogger()
    defer sugarLogger.Sync()
    simpleHttpGet("www.sogo.com")
    simpleHttpGet("http://www.sogo.com")
}

func InitLogger() {
    writeSyncer := getLogWriter()
    encoder := getEncoder()
    core := zapcore.NewCore(encoder, writeSyncer, zapcore.DebugLevel)

    logger := zap.New(core, zap.AddCaller())
    sugarLogger = logger.Sugar()
}

func getEncoder() zapcore.Encoder {
    encoderConfig := zap.NewProductionEncoderConfig()
    encoderConfig.EncodeTime = zapcore.ISO8601TimeEncoder
    encoderConfig.EncodeLevel = zapcore.CapitalLevelEncoder
    return zapcore.NewConsoleEncoder(encoderConfig)
}

func getLogWriter() zapcore.WriteSyncer {
    lumberjackLogger := &lumberjack.Logger{
        Filename:   "./test.log",
        MaxSize:   1,
        MaxBackups: 5,
        MaxAge:    30,
        Compress:   false,
    }
    return zapcore.AddSync(lumberjackLogger)
}

func simpleHttpGet(url string) {
    sugarLogger.Debugf("Trying to hit GET request for %s", url)
    resp, err := http.Get(url)
    if err != nil {
        sugarLogger.Errorf("Error fetching URL %s : Error = %s", url, err)
    } else {
        sugarLogger.Infof("Success! statusCode = %s for URL %s", resp.Status, url)
        resp.Body.Close()
    }
}
```

执行上述代码，下面的内容会输出到文件 `test.log` 中：

```
2019-10-27T15:50:32.944+0800    DEBUG    logic/temp2.go:48    Trying to hit GET
request for www.sogo.com
2019-10-27T15:50:32.944+0800    ERROR    logic/temp2.go:51    Error fetching URL
www.sogo.com : Error = Get www.sogo.com: unsupported protocol scheme ""
2019-10-27T15:50:32.944+0800    DEBUG    logic/temp2.go:48    Trying to hit GET
request for http://www.sogo.com
2019-10-27T15:50:33.165+0800    INFO     logic/temp2.go:53    Success! statusCode =
200 OK for URL http://www.sogo.com
```

同时，可以在 `main` 函数中循环记录日志，测试日志文件是否会自动切割和归档（日志文件每 1MB 会切割并且在当前目录下最多保存 5 个备份）。

至此，我们总结了如何将 Zap 日志程序集成到 Go 应用程序项目中。

翻译自 <https://dev-journal.in/2019/05/27/adding-uber-go-zap-logger-to-golang-project/>，为了更好地理解原文内容稍有更改。
